

Penetration Testing

ForeGate (Prediction Market)

Table of Contents

Executive Summary	4
Project Context	4
Penetration Test Scope	7
Security Rating	8
Severity Definitions	9
Status Definitions	10
Penetration Test Findings	11
Conclusion	29
Our Methodology	30
Disclaimers	32
About Hashlock	33

CAUTION

THIS DOCUMENT IS A SECURITY PENETRATION TEST REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE THAT COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR THE USE OF THE CLIENT.

Executive Summary

The Foregate team partnered with Hashlock to conduct a security Penetration Test of their Foregate Project code base. Hashlock manually and proactively reviewed the code to ensure the project's team and community that the deployed tools were secure.

Project Context

ForeGate is a next-generation prediction market platform, empowering users to forecast real-world events and trade on their outcomes. Whether you're a professional trader, industry expert, or just someone who follows trends, ForeGate offers a transparent, data-driven, and engaging marketplace where your insights can translate into real earnings.

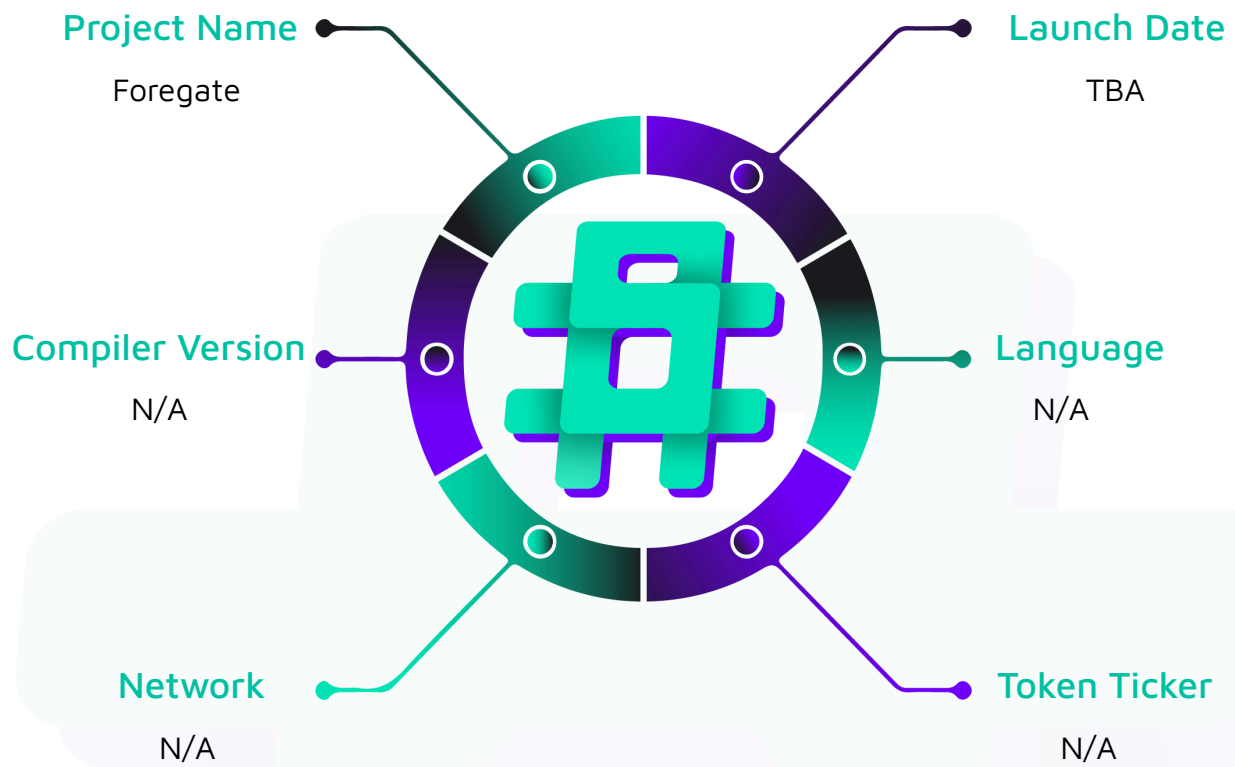
Project Name: Foregate

Project Type: Prediction Market

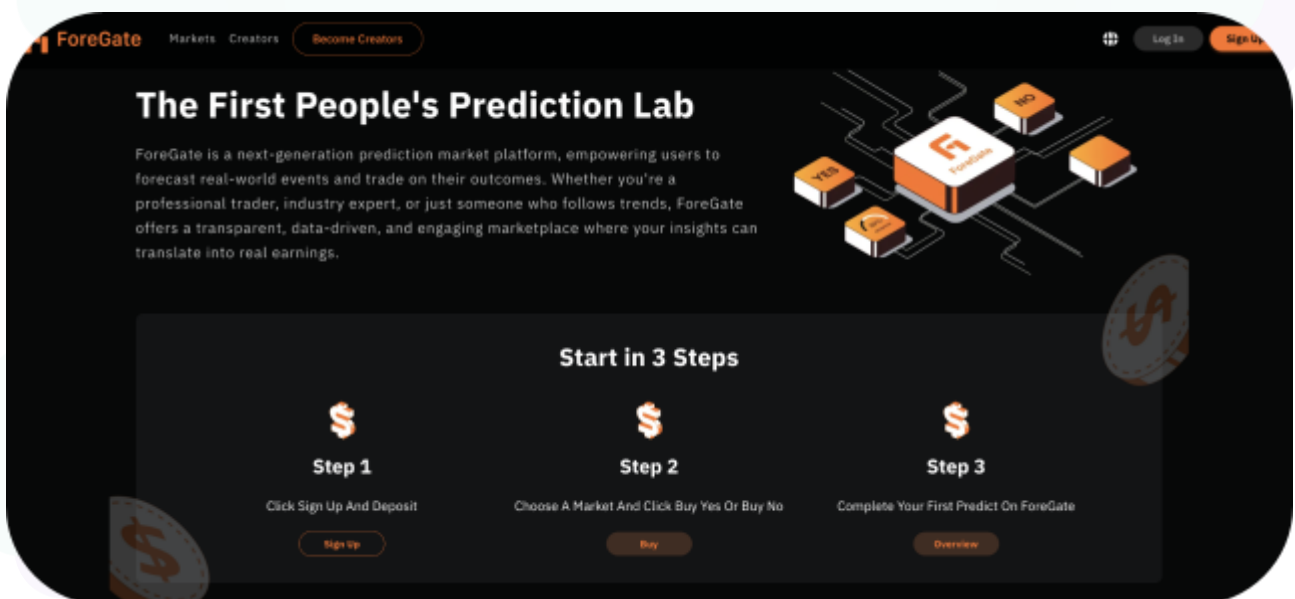
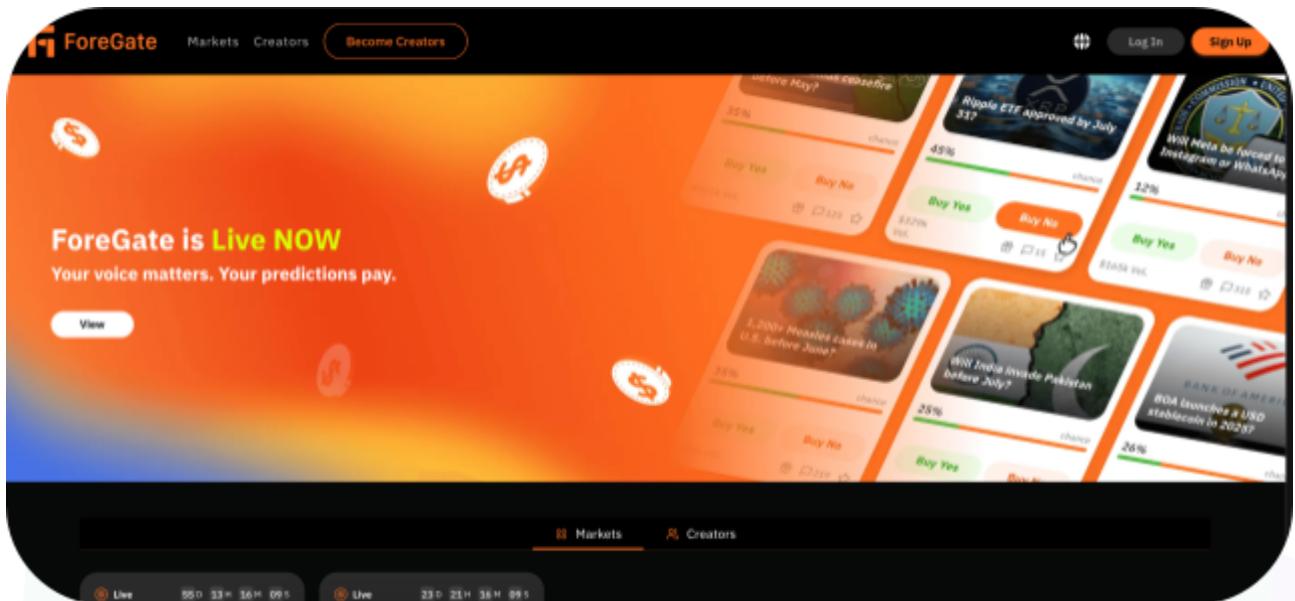
Website:

Logo:



Visualised Context:

Project Visuals:



Penetration Test Scope

At Hashlock, we executed a comprehensive penetration test on the Foregate project. Our scope included a detailed security assessment, where we rigorously examined the code to identify vulnerabilities and assess overall efficiency. The testing process involved a meticulous manual review, supplemented by advanced software-assisted techniques, to ensure thorough analysis and accuracy.

Description	Foregate Code Base
Platform	Web apps & APIs
Penetration Test Date	September, 2025
Repository/Website URL/IP Tested (1)	https://github.com/gph-tech/fg-backend/tree/release/1.2.0-rc%2B20250811
Repository/Website URL/IP Tested (2)	https://github.com/gph-tech/fg-frontend-user/tree/release/1.2.0-rc%2B20250811
Repository/Website URL/IP Tested (3)	https://github.com/gph-tech/fg-frontend-oms/tree/release/1.2.0-rc%2B20250812
Fix Review GitHub Commit Hash (1)	cdc221a8a8431bf6886eb222b79bdfb9c192ae9d
Fix Review GitHub Commit Hash (2)	88898c2d1c8a24e5b87b3e1adfc8aa93926f0cc1
Fix Review GitHub Commit Hash (3)	eeff183ca3f3d3f73ac91e6e707e4d551846ffbd

Security Rating

After Hashlock's Penetration Test, we found the code base to be **"Secure"**. The code follows simple logic, with correct and detailed ordering.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on-chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Penetration Test Findings](#) section.

All vulnerabilities initially identified have now been resolved.

Hashlock found:

3 High-severity vulnerabilities

2 Medium-severity vulnerabilities

7 Low-severity vulnerabilities

1 QA

Caution: *Hashlock's Penetration Tests do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are purely informational and don't impact functionality. These notes help clients improve the clarity, maintainability, or overall structure of the code, ensuring a cleaner and more efficient project. They should be addressed for optimization but are not critical to the system's performance or security.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Penetration Test Findings

High

[H-01] OSM APIS - Lack of authorization checks on admin endpoints

Description

API endpoints used by the administration tasks do not check if the user initiating the operation has the proper roles.

Vulnerability Details

The project includes administration endpoints located at devomsapis.foregate.com. These endpoints perform operations that normal users should not be able to. However, there are no checks to see if the user is indeed an admin or an operator. Normal users can call these endpoints without any problems. For example:

```
@PostMapping("/add")

@ApiOperation(value = "add a new market category")

public Result<Categories> addCategories(@Valid @RequestBody CategoriesDto
categoryDto, HttpServletRequest request) {

    Users user = (Users) request.getAttribute("user");

    // 设置创建人信息

    Categories category = new Categories();

    category.setName(categoryDto.getName());

    category.setDescription(categoryDto.getDescription());

    category.setNameZh(categoryDto.getNameZh());

    category.setSortNum(categoryDto.getSortNum());
```

```

        category.setIsOnline(categoryDto.getIsOnline());

        category.setLogoUrl(categoryDto.getLogoUrl());

        category.setCreatedBy(user.getId());

        category.setCreatedAt(LocalDateTime.now());

        // 插入数据库

        categoriesService.save(category);

        return new Result().success(category);
    }

```

Proof of Concept

We can demonstrate this problem by using a normal user's authorization cookie to edit a market:

```

POST /api/user/oms/market/application/edit HTTP/2

Host: devomsapis.foregate.com

Cookie:
Authorization=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJmb3JlZ2F0ZSI6InN1YiI6Im1oQ1lNckExZG5xZW1LNnNuRUxBcHdsR1RNT0Z1TXFXIiwiaWF0IjoxNzU3MDEzMDEzLCJleHAiOiJlE3NTcwOTk0NzN9.j8x0rrlaghJod2jSWa9TM0Jd7ezd-t4icgJ0GL3VKN4

Content-Length: 1535

Sec-Ch-Ua-Platform: "macOS"

Accept-Language: en-US,en;q=0.9

<...>

{"marketsVo":{"uuid":"395f7117-7f43-4f0a-ab86-bd59779ec459"},

<...>

```

And we get a success message:

```

HTTP/2 200 OK

Date: Fri, 05 Sep 2025 18:10:51 GMT

```

```
Content-Type: application/json

Vary: Accept-Encoding

Access-Control-Allow-Origin: https://omsdev.foregate.com

Access-Control-Allow-Credentials: true

Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS

Access-Control-Allow-Headers:
Content-Type,X-CAF-Authorization-Token,sessionToken,X-TOKEN,Authorization,user-account
,access-token,x-frame-options

Access-Control-Max-Age: 3600


{"code":200,"msg":"success",

<...>
```

Impact

Malicious users can use these APIs to create markets, steal funds by affecting market outcomes, or make themselves creators, among other things.

Recommendation

API endpoints that are admin-only should check if the user has the proper roles.

Status

Resolved

[H-02] MarketController#resolveMarketResult - Broken access control in resolveMarketResult

Description

Improper access control checks result in users other than the market creator being able to set the market result.

Vulnerability Details

The function resolveMarketResult does not check if the user sending the request is the actual owner of the market; the only check performed is whether the user is a creator or not.

```
public Result resolveMarketResult(HttpServletRequest request, @RequestBody
VerifyingMarketResultDTO verifyingMarketResultDTO) throws IOException {

    Users user = (Users) request.getAttribute("user");

    List roles = (List) request.getAttribute("roles");

    //      log.info("user roles:{",roles);

    if(roles.size() == 0){

    //      log.info("user roles.size():{",roles.size());

        return new Result().error(StatusCode.PERMISSION_DENIED);

    }else if(roles.contains(1L) || roles.contains(2L) || roles.contains(3L)||
roles.contains(5L)){

        if(verifyingMarketResultDTO.getOptions() == null){

            return new Result().error(StatusCode.OPTIONS_IS_REQUIRED);

        }

    }
```

Proof of Concept

By crafting a request like this:

```
POST /api/market/upd-market-result HTTP/2
```

```
Host: devtssapis.foregate.com
```



```
Cookie:
Authorization=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJmb3JlZ2F0ZSI6InN1YiI6IkJFOUwJ5VVd4SzFITENBY1BVYkd4aFR4dHE2NTdNaHBVbiwiWF0IjoxNzU2OTIwMjI3LCJleHAiOjE3NTcwMDY2Mjd9.3w-Vs13Ry0zdK7QrQeoJzh2vdXEAMXum6GC1brnBhzc

Accept-Encoding: gzip, deflate, br

Priority: u=1, i

{"marketId":"1029","resType":"Resolution","options":[{"outComeOptionId":"2163","isWin":1}],

"reasonText":"asd",

"fileUrl":"asdasd"

}
```

We can resolve a market that we do not own.

Impact

A malicious user can resolve markets in a way that benefits them, which can result in loss of funds for other users or a significant administrative workload.

Recommendation

The function should check whether the user sending the request is the owner of the market.

Status

Resolved

[H-03] AuthController#connectWallet - Wallet Authentication Accepts Arbitrary Messages

Description

Currently, the Foregate website accepts email-password and wallet authentication. Wallet authentication is facilitated by the user signing a message and sending both the message and the signature to prove ownership of the wallet. However, since the auth endpoint accepts any message-signature combo, attackers can use previously known combos to log in as other users.

Proof of Concept

Here we use a message and signature combo that we created:

```
POST /api/user/auth/wallet HTTP/2
Host: devtssapis.foregate.com
Cookie:
Authorization=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJmb3JlZ2F0ZSI6InN1YiI6Im1oQ1lNckExZG5xZW1LnN1R1NT0Z1TXFXIiwiaWF0IjoxNzU2MzY1NTAxLCJleHAiOiJlE3NTY0MjE5MDF9.6wpG86P9V77KL1qHZ1B9GUYFqpeRyAAybPQM7RYaWnU
Content-Length: 205
Accept-Encoding: gzip, deflate, br
Priority: u=4, i

{"address": "FamAAzW3qxs6krDcHdVRYUq1DUUrUZFSPPS4Qjy6ghzy", "message": "hello foregate", "signature": "3WKxYvutUKp2EphUuNvjFZbHPf6yTr1wt8sa2ecbvynos4g2C8U8DVGkEvCpEFeeFmitAbM9rWR2hBPnFXpYt97p2", "chain": "solana"}
```

And we are in:

```
{"code":200,"msg":"success","data":{"firstName":null,"lastName":null,"createdAt":"2025-08-27T19:32:50","is2FAEnabled":false,"language":"en-US","id":39586,"roleList":[{"id":39690,"userId":39586,"roleId":4}], "userName": "Wallet_1756323170", "isActive": true, "userId": "c8UbH6e7xXEdDszCSYZspBm040Ijensa", "email": null, "isEmailVerified": false}}
```

Impact

Attackers can use this vulnerability to log in as other users who have known message-signature combos and drain their funds.

Recommendation

When authenticating users by their signatures, a nonce value that is tracked by the server should be used to eliminate the possibility of signature replay attacks.

Status

Resolved

Medium

[M-01] MarketApplicationController#edit - Application is editable after it is sent

Description

Currently, applications are editable after they are sent for confirmation, which differs from the behaviour of the UI.

Vulnerability Details

Currently, the only check that is done when the edit function is called is whether the application has been approved.

```
if(creatorMarkets.getStatus().equals(ApplicationStatus.Approve.name())){  
    return new Result().error(StatusCode.CANT_EDIT_MARKET);  
}
```

Users can still edit their applications, even though it is not allowed on the UI, while it is waiting for confirmation using the edit endpoint.

Impact

Malicious users could use this unintended behaviour to get human operatives to confirm markets that they normally would not.

Recommendation

The functionality on the UI and the backend should be harmonized.

Status

Resolved

[M-02] MarketApplicationController#submit - Market applications allow non-standard fees

Description

While sending market applications, it is possible to set non-standard, very high fees.

Vulnerability Details

Normally, while creating a new market, the UI allows you to select from 1 to 5% fee amounts. However, the backend accepts any fee rate that is above 0.

Proof of Concept

Here we set a 100% fee:

```
POST /api/user/oms/market/application/submit HTTP/2
Host: devtssapis.foregate.com

<...>

"transFee":100,"liquidity":100,

<...>
```

It is accepted:

```
{"code":200,"msg":"success","data":null}
```

Impact

Malicious users can set very high fees, which would affect unaware users.

Recommendation

The backend should validate that the fee rate is acceptable.

Status

Resolved

Low

[L-01] NotificationsController#getUserNotifications - Broken access control when reading notifications

Description

The endpoint for reading user notifications accepts any user id and returns their notifications.

Proof of Concept

Using the session token of user 39581, we can read another user's notifications:

```
GET /api/user/notification/list?userId=39570 HTTP/2

Host: devtssapis.foregate.com

Cookie:
Authorization=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJmb3JlZ2F0ZSIsInN1YiI6Im1oQ1lNckExZG5xZW1LNnNuRUpBcHdsR1RNT0Z1TXFXIiwiaWF0IjoxNzU3MDEzMDEzLCJleHAiOiJlE3NTcwOTk0NzN9.j8x0rrlaghJod2jSWa9TM0Jd7ezd-t4icgJ0GL3VKN4

Sec-Fetch-Dest: empty

Referer: https://fgdev.foregate.com/

Accept-Encoding: gzip, deflate, br
```

The response:

```
{"code":200,"msg":"success","current":1,"size":10,"total":0,"pages":0,"data":[{"id":53185,"userId":39570,"messageType":0,"title":"Order Matched","titleZh":"您的掛單已成交", "icon":"https://foregate-ts.s3.amazonaws.com/upload/20250821/39571/market-banner-1755745476727-29my.png",
```

Recommendation

The notifications endpoint should only return information about the current user.

Status

Resolved

[L-02] UserQueryController - Queries do not honor privacy settings.**Description**

Some endpoints under `/user/query` do not honor privacy settings configured by users.

Proof of Concept

If we query the information using another api (volumeTradeall), we get obfuscated information:

```
{ "code": 200, "msg": "success", "data": { "positionValue": -9999, "profitOrLostValue": -9999, "userAddress": "*****", "userId": "39582", "volumeTradedAll": 75.0000000000, "creatorVolume": null, "creatorTrader": null, "creatorTotalMarkets": null, "creatorPNL": null, "userName": "User_1756239603", "avatarUrl": "https://raw.githubusercontent.com/jimcola99/corruptsvgfile/refs/heads/master/svgavatar.svg" } }
```

However, if we use `query/amount`, `query/profit`, `query/position`, `query/volume` endpoints, we get real values :

```
{ "code": 200, "msg": "success", "data": { "positionValue": 0.6536529373230000000000 } }

{ "code": 200, "msg": "success", "data": { "positionValue": 74.9036529373230000000000 } }

{ "code": 200, "msg": "success", "data": { "volumeTraded": 75.0000000000, "volumeTradedSell": 0, "volumeTradedBuy": 75.0000000000 } }
```

Recommendation

All endpoints should honor privacy settings..

Status

Acknowledged

[L-03] Image Fields - Remote resources can be added to markets and user profiles

Description

Users can bypass the upload links generated by Amazon S3 buckets and use remote resources on image fields.

Proof of Concept

Instead of the URL generated by the S3 service, we can add our own.

```
POST /api/user/me/profile/update HTTP/2

Host: devtssapis.foregate.com

Cookie:
Authorization=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJmb3JlZ2F0ZSIsInN1YiI6Im1oQ1lNckExZG5xZWlLnNuRUUpBcHdsR1RNT0Z1TXFXIiwiaWF0IjoxNzU2MjE0MTM2LCJleHAiOjE3NTYzMDA1MzZ9.isPdULI4DoAiX9X65b6FDd-7EPsEW_8y6gE3qdj806Y

Referer: https://fgdev.foregate.com/

Accept-Encoding: gzip, deflate, br

Priority: u=1, i

{"userName":"User_1756212803","bio":"Crypto enthusiast and trader","profilePicture":"https://xx.xx.xx.xx:9999/asd.png"}
```

Impact

Malicious actors can use images they host in order to gain information about other users or trick them.

Recommendation

All endpoints that accept an image URL should validate that it belongs to the S3 bucket owned by the project.

Status

Acknowledged

[L-04] - Remote resources can be added to markets

Description

Users can bypass the upload links generated by Amazon s3 buckets and use remote resources on image fields..

Proof of Concept

If we query the information using another api (volumeTradeall), we get obfuscated information:

```
{ "code": 200, "msg": "success", "data": { "positionValue": -9999, "profitOrLostValue": -9999, "userAddress": "*****", "userId": "39582", "volumeTradedAll": 75.0000000000, "creatorVolume": null, "creatorTrader": null, "creatorTotalMarkets": null, "creatorPNL": null, "userName": "User_1756239603", "avatarUrl": "https://raw.githubusercontent.com/jimcola99/corruptsvgfile/refs/heads/master/svgavatar.svg" } }
```

However, if we use `query/amount`, `query/profit`, `query/position`, `query/volume` endpoints, we get real values :

```
{ "code": 200, "msg": "success", "data": { "positionValue": 0.6536529373230000000000 } }  
  
{ "code": 200, "msg": "success", "data": { "positionValue": 74.9036529373230000000000 } }  
  
{ "code": 200, "msg": "success", "data": { "volumeTraded": 75.0000000000, "volumeTradedSell": 0, "volumeTradedBuy": 75.0000000000 } }
```

Recommendation

All endpoints should honor privacy settings..

Status

Resolved

[L-05] - Unrestricted upload to S3 bucket

Description

The S3 bucket upload system accepts any kind of file.

Proof of Concept

Uploading an exe file:

```
POST /api/user/client/aws-s3/sign-upload HTTP/2
Host: devomsapis.foregate.com

<..>

{"fileType":"application/x-msdownload","s3FileName":"market-1757098793501-1.exe"}
```

Success:

```
{"code":200,"msg":"success","data":{"uploadTempUrl":
```

Impact

Accepting any kind of file might be problematic as malicious actors can use foregate servers to spread malware or other kinds of illegal files.

Recommendation

Only file extensions that are allowed should be accepted in order to limit the risks of allowing user uploads.

Status

Acknowledged

[L-06] MarketApplicationController#edit - Broken access control in rejected application editing

Description

After their market applications are rejected, users can edit and resend them. However, due to a lack of checks regarding who owns the application, any user can edit rejected applications. The edit endpoint also lacks checks for user roles. So any user, regardless if they are creators or not, can edit rejected applications.

Proof of Concept

Here is an edit request sent by a user who does not own the application, changing the name:

```
POST /api/user/oms/market/application/edit HTTP/2
Host: devtssapis.foregate.com

<...>

{"marketsVo":{"uuid":"1edf00cf-275c-4183-9a83-4211f7906979","status":"Approved","reason":"sdaadasd","marketId":null,"title":"SonicXX",
```

We get success:

```
HTTP/2 200 OK

<...>

{"code":200,"msg":"success","data":{
```

Impact

Malicious actors can edit other users' market applications to affect the normal functioning of the website.

Recommendation

The edit endpoint should do proper checks to see if the user owns the application and if they are indeed a creator.

Status

Acknowledged

[L-07] MarketApplicationController#cancel - Broken access control in rejected application canceling

Description

After their market applications are rejected, users can choose to delete them instead of editing them. However, due to a lack of checks regarding who owns the application, any user can delete rejected applications.

Proof of Concept

Here is a delete request sent by a user who does not own the application, changing the name:

```
PUT /api/user/oms/market/application/cancel?uuid=203453ee-b1c9-4713-a872-feeeec889c86
HTTP/2

Host: devtssapis.foregate.com

<...>
```

We get success:

```
HTTP/2 200 OK

Date: Tue, 26 Aug 2025 22:37:09 GMT

<...>

{"code":200,"msg":"success","data":null}
```

Impact

Malicious actors can delete other users' market applications to affect the normal functioning of the website.

Recommendation

The cancel endpoint should do proper checks to see if the user owns the application and if they are indeed a creator.

Status

Resolved



QA

[Q-01] Project Wide - website can be used without verifying email

Description

Although the UI does not work without verifying your email address the backend does not check this. This could lead to impersonation or spam signups.

Recommendation

Add checks for email verification before core website functionality can be used.

Status

Acknowledged

Conclusion

After Hashlock's analysis, the Foregate project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our Penetration Tests a collaborative effort. The objective of our security Penetration Tests is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security Penetration Test process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future Penetration Tests. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the code base under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other Penetration Test results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we still need to verify the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown not to represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed the code bases in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the code base source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the Penetration Test makes no statements or warranties on the security of the code. It also cannot be considered a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this code base.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Code is deployed and executed on a platform. The platform, its programming language, and other software related to the code base can have their own vulnerabilities that can lead to attacks. Thus, the Penetration Test can't guarantee the explicit security of the Penetration Tested code bases.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.

